Guia de Resolução de Problemas: Git e GitHub

Autor: Jeffa08P
Data: Março de 2026
Versão: 1.0

Introdução

Este documento reúne os principais problemas relacionados ao Git e GitHub que surgiram durante o desenvolvimento do projeto "numero-secreto-alura", com as respectivas soluções aplicadas. O objetivo é servir como referência rápida para situações similares no futuro.

1. Erro ao Enviar Código para o GitHub: "Updates were rejected"

Situação

Ao tentar enviar o código local para o repositório remoto usando:

git push -u origin main

O Git retornou o seguinte erro:

! [rejected] main -> main (fetch first)
error: failed to push some refs to 'github.com:jeffa08P/numero-secreto-alura'
hint: Updates were rejected because the remote contains work that you do not have
hint: locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.

Causa

O branch `main` no GitHub já continha commits que não existiam no repositório local. Isso geralmente acontece quando o repositório é inicializado no GitHub com um README ou .gitignore, criando um primeiro commit que não existe localmente.

Solução Aplicada

1. Baixar o histórico remoto e permitir a junção de históricos não relacionados:

   ```
   git pull origin main --allow-unrelated-histories
   ```
2. Resolver conflitos caso apareçam (no seu caso, não houve conflitos).
3. Adicionar arquivos se necessário e criar commit de merge:

   ```
   git add .
   git commit -m "Merge histories"
   ```
4. Enviar para o GitHub:

   ```
   git push -u origin main
   ```

Resultado

O branch `main` local foi sincronizado com o remoto, e o push foi realizado com sucesso. A mensagem final mostrou:

```
branch 'main' set up to track 'origin/main'.
```

2. Remover Arquivos Específicos do Repositório

Situação

Havia arquivos indesejados já versionados e visíveis no GitHub que precisavam ser removidos:

- Pasta `img/`
- Arquivo `app.js`
- Arquivo `index.html`

- Arquivo `style.css`

Objetivo

Apagar esses arquivos tanto do repositório local quanto do GitHub.

Solução

1. Remover os arquivos do controle de versão:
   git rm -r img
   git rm app.js index.html style.css
   Alternativamente, você pode apagar os arquivos manualmente pelo explorador e depois usar `git add .` para registrar a remoção.
2. Criar commit com a remoção:

   git commit -m "Remove arquivos antigos"
3. Enviar para o GitHub:

   git push origin main

Resultado

Os arquivos foram removidos da listagem do GitHub. Eles permanecem no histórico de commits, o que é normal e permite recuperação se necessário.

3. Arquivos Não Rastreados (Untracked Files)

Situação

O comando `git status` mostrou:

Untracked files:
(use "git add <file>..." to include in what will be committed)
"djkjasdlk\30ß\247fjlsakd"

nothing added to commit but untracked files present

Significado

Arquivo não rastreado é aquele que existe na pasta do projeto mas nunca recebeu `git add`. Ele não será incluído em commits nem enviado ao GitHub a menos que seja explicitamente adicionado.

Soluções Possíveis

Opção 1: Apagar manualmente

Navegue até a pasta do projeto no explorador de arquivos ou VS Code e delete o arquivo.

Opção 2: Usar comando Git para limpar arquivos não rastreados

```
git clean -f
```

Este comando remove **todos** os arquivos não rastreados da pasta. Use com cuidado.

Opção 3: Ignorar o arquivo

Crie ou edite o arquivo `.gitignore` e adicione o nome do arquivo que deseja ignorar permanentemente.

4. Desfazer git add (Remover do Stage)

Situação

Após executar `git add`, vários arquivos apareceram como "Changes to be committed" (em verde), mas você decidiu não commitá-los ainda.

Objetivo

Tirar os arquivos do stage (área de preparação) sem apagá-los do disco.

Solução

```
git reset
```

Este comando remove todos os arquivos do stage, mas mantém as alterações no disco. Os arquivos voltam ao estado "untracked" ou

"modified".

Caso Queira Também Apagar os Arquivos

Se além de tirar do stage você quiser apagar os arquivos/pastas:

```
git rm -r nome_da_pasta
```

Ou simplesmente delete manualmente pelo explorador.

5. Sincronizar com o GitHub (Descartar Mudanças Locais)

Situação

Você fez mudanças locais ou commits locais que deseja descartar, voltando o repositório local para o estado exato do GitHub.

Solução

1. Baixar o estado mais recente do remoto:

   ```
   git fetch origin
   ```
2. Forçar o branch local a ficar idêntico ao remoto:

   ```
   git reset --hard origin/main
   ```

Atenção

O comando `git reset --hard` **apaga permanentemente**:

- Commits locais que não foram enviados ao GitHub
- Modificações não commitadas nos arquivos

Use apenas quando tiver certeza de que não precisa dessas mudanças.

6. Apagar o Último Commit

Cenário 1: Commit Ainda Não Foi Enviado ao GitHub

Opção A: Manter as alterações dos arquivos no disco (só desfaz o commit)

git reset --soft HEAD~1

Após este comando, os arquivos continuam modificados e no stage, prontos para um novo commit.

Opção B: Apagar também as alterações dos arquivos

git reset --hard HEAD~1

Isso volta os arquivos ao estado do commit anterior, descartando todas as mudanças.

Cenário 2: Commit Já Foi Enviado ao GitHub

1. Primeiro, desfaça o commit localmente usando uma das opções acima (`--soft` ou `--hard`).
2. Depois, sobrescreva o histórico remoto:

   git push origin main --force

Aviso Importante

O uso de `--force` reescreve o histórico do GitHub. Evite fazer isso em repositórios compartilhados com outras pessoas, pois pode causar conflitos graves.

7. Apagar Único Commit do Histórico Local

Situação

O comando `git log --oneline` mostrou apenas um commit:

ff9805a (HEAD -> main) backup

Este era o único commit do branch local, e você queria removê-lo.

Solução 1: Manter o GitHub e Descartar o Commit Local

Se o GitHub tem o histórico correto e você quer apenas descartar esse commit local:

```
git fetch origin
git reset --hard origin/main
```

Isso faz o `main` local voltar a ser exatamente igual ao `main` do GitHub.

Solução 2: Zerar Tudo (Inclusive no GitHub)

Se você quer apagar esse commit tanto localmente quanto no GitHub, recriando o branch do zero:

1. Criar um novo branch órfão (sem histórico):

   ```
   git checkout --orphan temp
   ```
2. Limpar todos os arquivos rastreados:

   ```
   git rm -rf .
   ```
3. Criar um commit vazio inicial:

   ```
   git commit --allow-empty -m "Initial empty commit"
   ```
4. Renomear o branch para main:

   ```
   git branch -M main
   ```
5. Sobrescrever o GitHub:

   ```
   git push origin main --force
   ```

Resultado

O branch `main` é recriado sem histórico anterior, e o commit "backup" é completamente removido.

8. Resumo de Comandos Úteis

Comando	Função
git status	Verificar estado do repositório
git log --oneline	Ver histórico resumido de commits
git add .	Adicionar todos os arquivos ao stage
git commit -m "msg"	Criar commit com mensagem
git push origin main	Enviar commits para o GitHub
git pull origin main	Baixar e mesclar mudanças do GitHub
git fetch origin	Baixar mudanças sem mesclar
git reset	Tirar arquivos do stage
git reset --hard origin/main	Sincronizar com GitHub (descarta local)
git reset --soft HEAD~1	Desfazer último commit (mantém arquivos)
git reset --hard HEAD~1	Desfazer último commit (apaga alterações)
git rm arquivo	Remover arquivo do Git
git rm -r pasta	Remover pasta do Git
git clean -f	Apagar arquivos não rastreados
git push origin main --force	Sobrescrever histórico remoto

Table 1: Comandos Git utilizados na resolução de problemas

Conclusão

Este guia documenta as principais situações enfrentadas durante o trabalho com Git e GitHub no projeto "numero-secreto-alura". Os comandos e soluções apresentados podem ser aplicados em contextos similares no futuro.

Lembre-se sempre de:

- Verificar o estado do repositório com `git status` antes de executar comandos destrutivos
- Ter cuidado com `--force` em repositórios compartilhados

- Fazer backup de trabalhos importantes antes de usar `git reset --hard`
- Consultar `git --help` ou a documentação oficial quando tiver dúvidas

Referências

- Documentação oficial do Git: https://git-scm.com/doc
- GitHub Docs: https://docs.github.com
- Pro Git Book (gratuito): https://git-scm.com/book/en/v2